(5px), bottom padding (20px), top border (2px), and bottom border (4px). The total height is therefore 50px + 5px + 20px + 2px + 4px = 81px.

When `box-sizing: border-box` is applied, the specified width and height (100px and 50px) include the content, padding, and border. This means the total width is 100px and the total height is 50px, regardless of the padding and border values. This approach greatly simplifies layout calculations and helps ensure consistent sizing across elements, making it a widely recommended practice for modern layouts.

However, exceptions do exist: precise positioning may require `box-sizing: content-box`, especially with `position: relative` or `position: absolute`. Legacy code or third-party libraries may depend on the default `content-box` behavior, and some complex layouts may need content-only sizing. Additionally, universal selectors applying `border-box` might introduce minor overhead in rare cases. Understanding these best practices and exceptions helps candidates stand out in discussions of adaptable CSS.

To handle a legacy codebase that uses `box-sizing: content-box` while migrating to `box-sizing: border-box` for new components, a gradual and controlled approach is recommended. Start by applying `box-sizing: border-box` selectively to new components or specific sections using targeted CSS rules or component-based selectors, rather than a global reset. This prevents unexpected layout shifts in legacy components, which may depend on the default `content-box` calculation for their sizing and spacing. Testing and validation are essential to ensure consistency across browsers.

Margins create transparent space around elements, separating them from others. Negative margins in the CSS box model let elements move closer or overlap others, altering their expected position. This can help with advanced layouts, spacing fixes, or visual effects requiring precise control. However, negative margins may cause unpredictable layouts, complicate debugging, and hide deeper structural issues if used as a workaround. Instead of negative margins, use Grid, Flexbox, transforms, padding, or positioning for more predictable and maintainable layouts.

Vertical (top and bottom) margins of certain elements can collapse—combining into a single margin rather than adding together. This occurs